

Revising the n_TOF PSA configuration for the X17 EAR2 campaign

What the original production reconstruction gets wrong, and what the revision measures instead

X17 / DREAM group

2026-07-28 run 224572

Abstract

The official Pulse Shape Analysis configuration for the 2026 X17 EAR2 campaign mis-identifies the γ -flash on the plastic scintillators in **37–85 % of bunches**, and times the flash on the SiPM walls from the transient of their own protection gate rather than from the flash itself. Since every hit’s physics time is $\text{tof} - t_{\text{flash}}$, the first defect shifts whole sets of hits by up to $11.6\,\mu\text{s}$, and the second puts a fixed $\sim 350\,\text{ns}$ offset between detectors that should agree. Together they made a hardware wall $\wedge$ plastic coincidence look as though the plastic fired only half the time.

Both are configuration faults, not detector faults: the raw waveforms contain the flash, unmissably, in every bunch. This note documents the revision, the measurements behind each change, and what it costs. On the reference run the flash mis-identification goes to **0.0 % on 12 of 13 trees**, the per-arm coincidence offsets go from $-362/+20/-333/-336\,\text{ns}$ to $+1.5/+2.0/+1.0/-2.0\,\text{ns}$, and our DREAM coincidence matcher improves from 93.7 % to **96.4 %** efficient — the latter measured with our offline repair switched off, so it tests the processing alone.

1 Why this was worth chasing

The X17 measurement pairs the n_TOF scintillator array with the DREAM detector. Its trigger is a hardware AND of a SiPM wall and a plastic scintillator in the same arm, so every triggered event must have a plastic partner. In the official reconstruction only about half of them did.

The explanation is not in the detectors. It is that $t_{\text{since flash}} = \text{tof} - t_{\text{flash}}$ is only as good as the PSA’s flash identification, and on the plastics that identification fails in most bunches.

2 Defect 1: the plastic flash is mis-identified

The plastic row of the UserInput sets `G-FLASH THRESHOLD = 50.` with no lower time limit. The plastic flash saturates the digitiser — it drives the signal from a baseline near 30 800 past zero — so 50 channels is about 0.2 % of the feature it is meant to find. Whatever noise excursion happens first in the $11.6\,\mu\text{s}$ before the flash wins instead.

The fix is to give the finder a threshold matched to the feature and a lower time limit: `G-FLASH THRESHOLD = 2000/1e4`. The flash sits at $11.63\,\mu\text{s}$ in every bunch by hardware, so refusing to look before $10\,\mu\text{s}$ removes the entire failure mode. The plateau is wide — 500, 2000 and 5000 give identical results on 851 channel–bunch combinations — so the choice is not delicate.

3 Defect 2: the walls time their own protection gate

The SiPM wall signal is blanked by a $\sim 1\,\mu\text{s}$ protection gate around the flash. The walls therefore never see the flash directly. What the digitiser records, in order, is: the gate *closing* at $11.24\,\mu\text{s}$; a

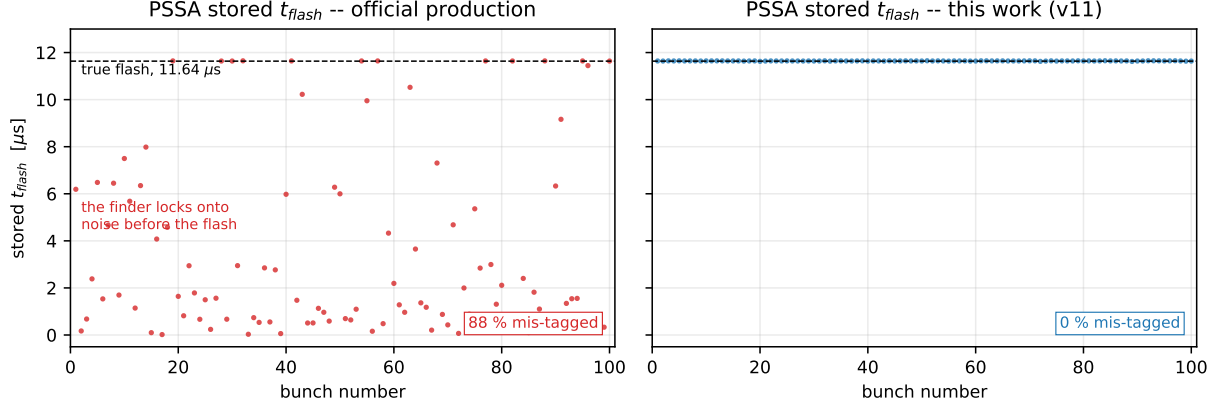


Figure 1: Stored t_{flash} for each bunch of tree PSSA. *Left:* the official production. The true flash is the dashed line at $11.64 \mu\text{s}$; most bunches are tagged somewhere in the noise before it, some by more than $11 \mu\text{s}$. *Right:* the same bunches after the revision. Each mis-tagged bunch shifts *all* of its hits by the tagging error.

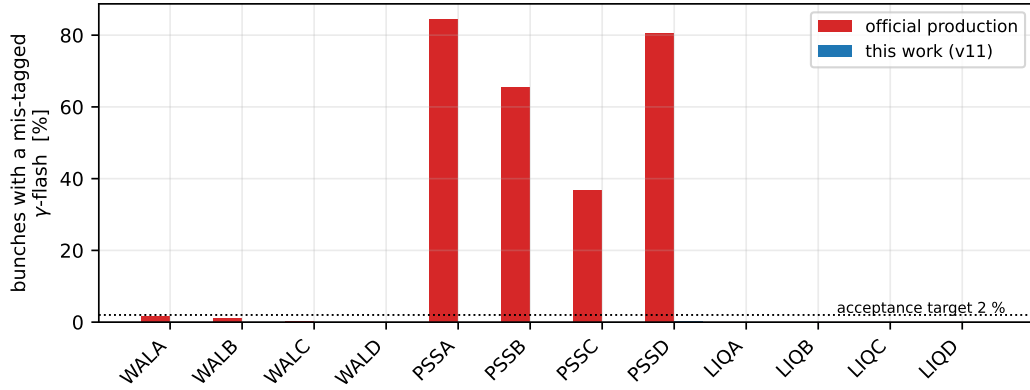


Figure 2: Fraction of bunches whose stored t_{flash} is more than 150 ns from that tree's own per-run mode, over 421 bunches. The liquids and the pickup are already clean, which is what shows the plastics' failure is configuration and not beam.

clamped copy of the real flash leaking through the closed gate at $11.60\ \mu\text{s}$; and the gate *opening* at $12.3\ \mu\text{s}$ with a rail-to-rail transient whose baseline takes tens of microseconds to recover.

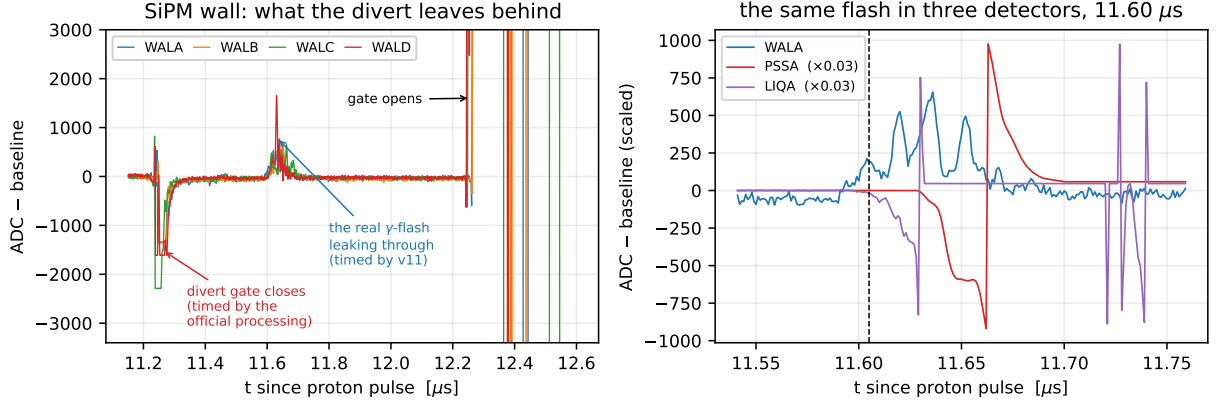


Figure 3: Raw stream1 waveforms, bunch 161. *Left:* one channel of each wall through the gate window. *Right:* the leak seen by the wall next to the same flash in a plastic and a liquid, neither of which is gated (scaled by 0.03). All three rise at $11.60\ \mu\text{s}$: the wall feature really is the flash.

With `G-FLASH THRESHOLD = 500.` and no time limit, which of the first two features wins is decided by their relative amplitude — and WALB’s gate transient is the weakest of the four walls, so WALB usually lands on the flash while WALA/C/D land on the gate. **That bistability is the origin of the $\sim 350\ \text{ns}$ cross-detector offset**, and it is why arm B looked innocent while the other three did not.

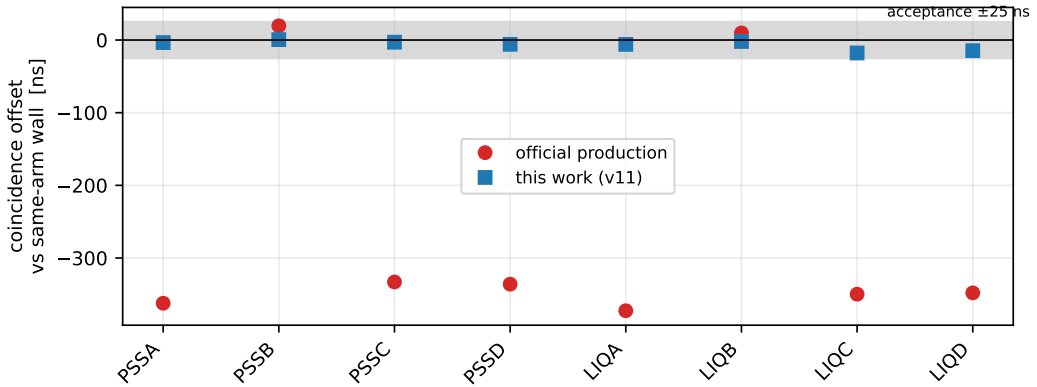


Figure 4: Position of the prompt coincidence peak between a large hit in each plastic or liquid and a hit in the same arm’s wall. A genuine coincidence should sit at zero. In the official processing three arms out of four are displaced by the gate’s lead time.

The fix is a lower time limit placed between the two features: `G-FLASH THRESHOLD = 250/11400.` The threshold must stay at or below 400: at 600 the weakest channels miss the leak entirely and fall through to the gate-opening transient, at which point the spread explodes from $\sim 30\ \text{ns}$ to $\sim 660\ \text{ns}$.

What the divert costs regardless. After the gate reopens the wall baseline is $+21\ 500$ channels at $13\ \mu\text{s}$ and still $+400$ at $30\ \mu\text{s}$. Wall hits are unusable from about 11.6 to $13\text{--}15\ \mu\text{s}$ and degraded to $\sim 40\ \mu\text{s}$ — E_n above roughly $1\ \text{keV}$ at the $19.5\ \text{m}$ flight path. This is a property of the hardware, not of the configuration, but it belongs in any flux or E_n analysis.

4 Elimination windows that removed real pulses

Table 1: Area/amplitude of isolated late-time pulses, measured directly from the raw waveforms, against the shipped elimination window.

family	measured p1 ... p99	shipped cut	fraction removed
WAL	42 ... 110	10 ... 200	0 %
PSS	1.3 ... 29	2 ... 20	~25 %
LIQ	1.7 ... 14	2 ... 10	~19 %

The wall cut sits a factor two above the largest value the walls ever produce, which is what a safe elimination window looks like; the plastic and liquid cuts sit inside the bulk of their own distributions (Fig. 5). Both were widened to 1 ... 60. The PSA guide is explicit that elimination should be loose and that final rejection belongs in the offline analysis.

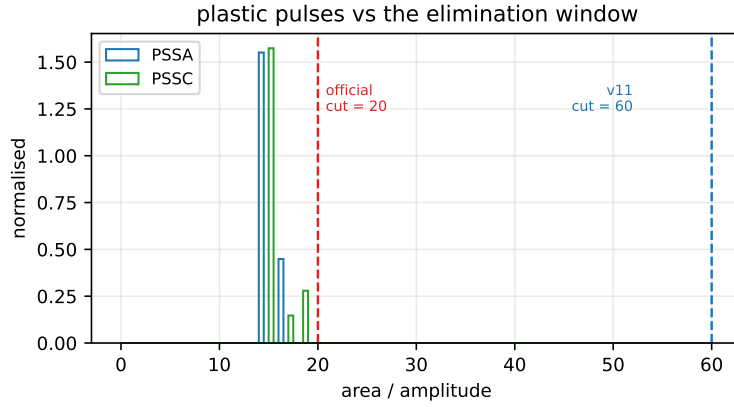


Figure 5: Area/amplitude of plastic pulses in the revised processing, with the old and new upper cuts marked. Everything to the right of the red line was being discarded.

5 Pulse shapes

5.1 The walls: the shipped templates are too short

Each shipped wall template is a *single raw pulse*, 314 ns long. The wall pulse is still at 4–6 % of its peak 200 ns after the maximum and 0.5 % at 500 ns, so the template ends inside the tail (Fig. 6). Every fitted amplitude then mis-accounts that tail, and the pileup deconvolution is working with a truncated kernel.

Table 2: Median fit χ^2 per wall, shipped vs measured templates.

	WALA	WALB	WALC	WALD
shipped (single pulse, 314 ns)	0.896	1.227	1.133	1.184
measured (median, 720–861 ns)	0.851	1.002	1.002	1.062

5.2 The plastics: fitting enabled, and it was the largest single gain

The plastics used `AMPLITUDE OPTION = 1` — a parabolic fit to the top, no deconvolution at all — while being the highest-rate tree in the file. Switching to shape fitting with measured 101 ns templates produced the biggest downstream improvement of any change, and by an instructive

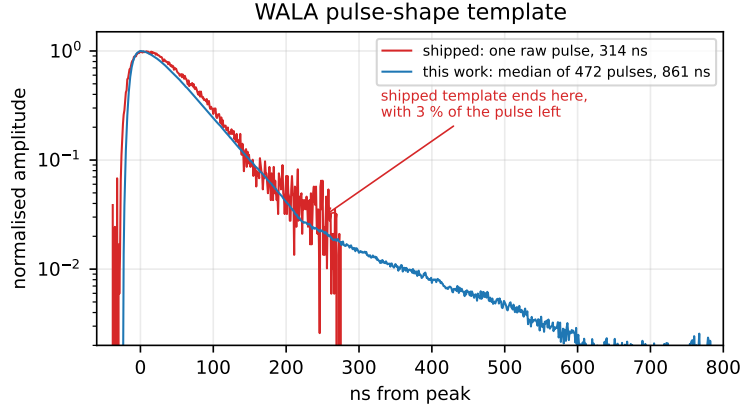


Figure 6: WALA pulse-shape template, normalised, logarithmic. The replacement is the median of 472 clean isolated pulses and runs to 861 ns.

route: it yields *fewer* plastic hits (0.72–0.99 of the previous count at every amplitude cut) but *better-timed* ones, merging pileup fragments back into a single correctly-timed pulse. The number of valid wall $\wedge$ plastic candidates rises even as the number of plastic hits falls.

5.3 The liquids: three attempts, all worse

We tried replacing the liquid templates three times — 551 ns averages, 81 ns averages, per-detector rather than shared — and every attempt was worse, with χ^2 more than doubling and amplitudes falling $\sim 30\%$. The shipped liquid pair spans FWHM 1 ns and 7 ns, a near-delta plus a normal pulse, and we suspect that spread does real work that a set of similar averaged shapes cannot reproduce. **The shipped liquid templates are kept.** We record the failure because it is the kind of change that looks obviously good and is not.

6 What the revision achieves, and what it costs

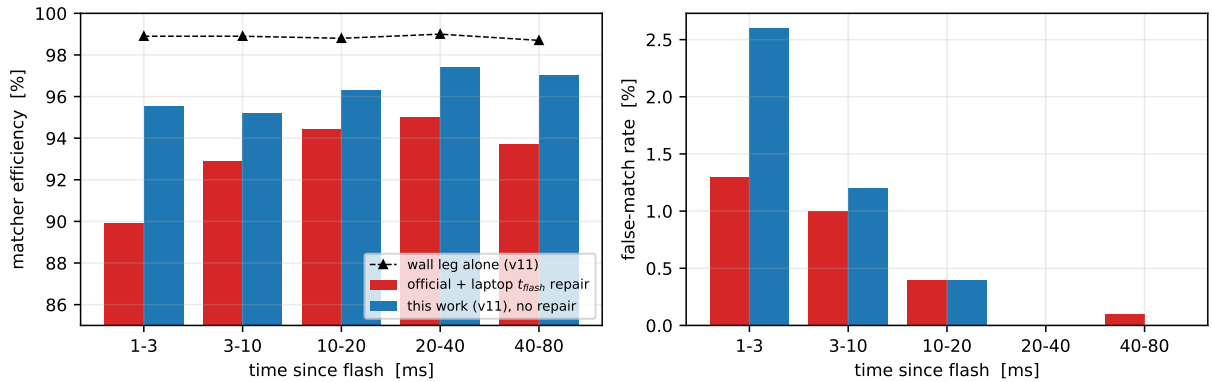


Figure 7: DREAM coincidence matcher, thresholded wall $\wedge$ plastic singles. *Left:* efficiency per time-since-flash bin; the triangles are the wall leg alone, which shows the plastic leg is what limits the AND. *Right:* the false-match rate, measured with a $+100\ \mu\text{s}$ time-shifted control. The official column already includes our offline t_{flash} repair — with the raw official file the matcher is 12.7% efficient and unusable.

The lower half of that table is the guard: it would be easy to gain hits by reconstructing them worse, so timing resolution, amplitude linearity and MIP width were tracked alongside efficiency. All are unchanged within a few percent.

Table 3: Summary on run 224572, DREAM run_79/stat090_0000. The revised column is measured with our offline t_{flash} repair *disabled*.

	official production	this revision
flash mis-identification	PSS 37–85 %	0.0 % on 12 of 13 trees
per-arm coincidence offset	−362/+20/−333/−336 ns	+1.5/+2.0/+1.0/−2.0 ns
matcher efficiency	93.7 % (<i>needs our repair</i>)	96.4 %
matcher false rate	0.5 %	0.6 %
in the 1–3 ms bin	89.9 % / 1.3 %	95.5 % / 2.6 %
wall timing, top↔bottom σ	6.65 ns → 6.65 ns	
wall↔plastic coincidence σ	6.46 ns → 6.41 ns	
MIP peak width (FWHM/peak)	1.22 → 1.22	
timing walk over amplitude	1.38 ns → 1.48 ns	

The cost, stated plainly. The false-match rate in the 1–3 ms bin roughly doubles, from 1.3 % to 2.6 %. That is the direct consequence of recovering the plastic hits: the candidate rate rises from ~ 935 to ~ 1042 per bunch. For this analysis it is a good trade, because efficiency in the same bin rises by 5.6 points and accidentals can be subtracted offline with a time-shifted control. An analysis that needs early-time purity instead can trade it back by tightening AREA/AMP HIGH towards 20.

Where the limit now is. The wall leg alone reaches 98.9 % and is flat in time; the AND reaches 96.4 %. The plastic leg therefore still costs 2.5 % overall and 3.4–3.7 % at early times. We established that this is *not* a pulse-recognition problem: three further variants with materially different plastic reconstructions — ± 40 % in amplitude, ± 30 % in hit count — all return exactly 96.4 %, because above ~ 2000 ADC they agree to better than 1 % and the hardware discriminator never sees anything below that. Further gains are analysis-side: the per-arm discriminator threshold model, the channel selection, dead time, or real detector inefficiency.

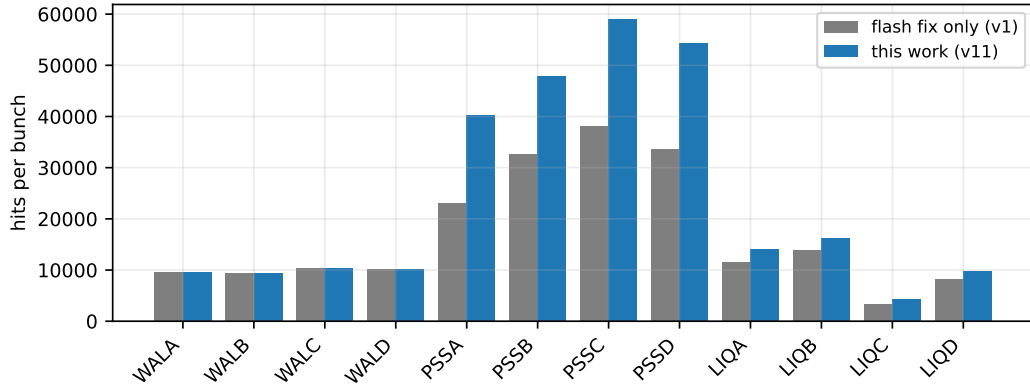


Figure 8: Hits per bunch per tree, flash fix alone versus the full revision. The walls are unchanged by construction — their elimination windows were already safe — which is the control that the change did what it was meant to and nothing else.

7 Reproducing this

All tooling is in the group repository under `ntof_processing/`: `grade_candidate.py` (flash identification, coincidence offsets, hit counts), `compare_fits.py` (fit χ^2), `quality_metrics.py` (timing resolution, amplitude linearity, MIP width, timing walk — all accidental-subtracted), `dream_regression.py` (the matcher), `make_variants.py` (generates a UserInput as a base plus

named column edits), `flash_finder_emulator.py` (pre-validates G-FLASH parameters on raw waveforms without a processing round-trip), and `make_pulse_shapes.py` (builds the templates). The numbers quoted here are in `report/results.json` with the tool that produced each one.

Two traps worth passing on. Coincidence widths must be accidental-subtracted: at these rates the nearest-hit-in-window distribution is dominated by chance, and without an off-time sideband subtracted the wall↔plastic width reads 38.8 ns instead of 6.4 ns. And the merge step of `RunProcessing.sh` failed on every EAR2 run we tried, with `max total download bytes exceeded` (max=1024 MB) — the per-file processing always succeeded, so we read the partials in `completed/` directly.